Flag feature

Status	done

Executive Summary
Assumptions
Functional requirements
Architectural principles (non functional)
- Proposed Implementation
  - Arch diagram - high level
  - Data model
  - Load balancing Strategy
  - Archiving and approval process
  - Rollback strategy
  - API vs SDK
  - Creating new users/groups
- 4 Metrics & Dashboards
- 7 Potential Impact and Dependencies
- 8 Unresolved questions

Executive Summary

A feature flag system that will allow developers to easily manage access to parts of their application to decouple go-live dates from deployment of code and to allow incremental delivery of work.

Assumptions

- Primary user group are developers/a lead engineer
- This feature flag is being built such that different customers from different companies can use the feature flag application. It is NOT an internal tool for

one organisation

- A feature can be mapped to many flags
- There will be a UI to make the feature flag

Functional requirements

- As a developer I want to manage features in my applications dynamically without deploying new code.
- As a developer I want to be able to choose different types of feature flags
- As a developer I want to be able to switch on feature flags for different users, groups or environments
- As a developer, I want to be notified if a feature flag has failed to disable/enable a feature.
- As a developer, I want to be able to have a percentage rollout of my features
- As a developer, I want to be able to managed the status of my flag rules (Active , inactive
- As a developer I want to be able to rollback a feature if it's being switched off
- As a lead engineer, I need to approve rollback of features
- As a user of applications, I need to approve the rollout of new features before they go-live.
- As a lead engineer/application owner, I need to approve the 'switching on' of feautures

Architectural principles (non functional)

- Extensible: If the number of flag types/groups/users increases, it shouldn't be difficult to add new mappings
- Highly available and low-latency: The flag system should always be available to check whether a feature is accessible to a group/user. The time taken to check whether the feature should be allowed is minimal.

- Configurable: The process to switch features on/off shouldn't be overly complex

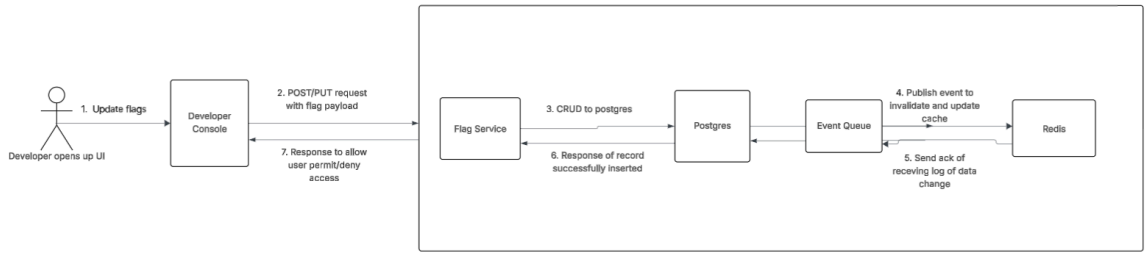
Proposed Implementation

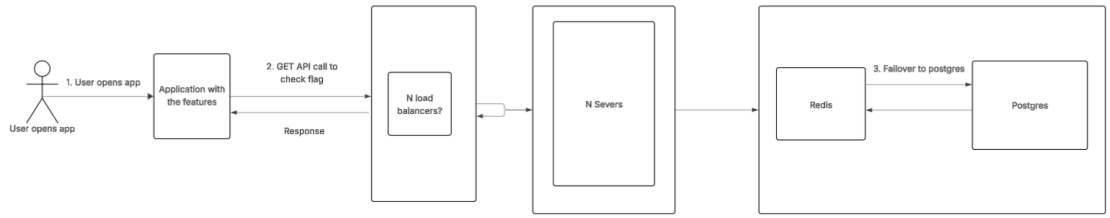
NOTE: I think I need some in betweens api gateways, load balancers and a mechanism to log?? not sure though

This is a read-heavy and data-heavy system as there is an underlying assumption that the number of users making requests to a feature is more frequent than the rate of writing new flags.

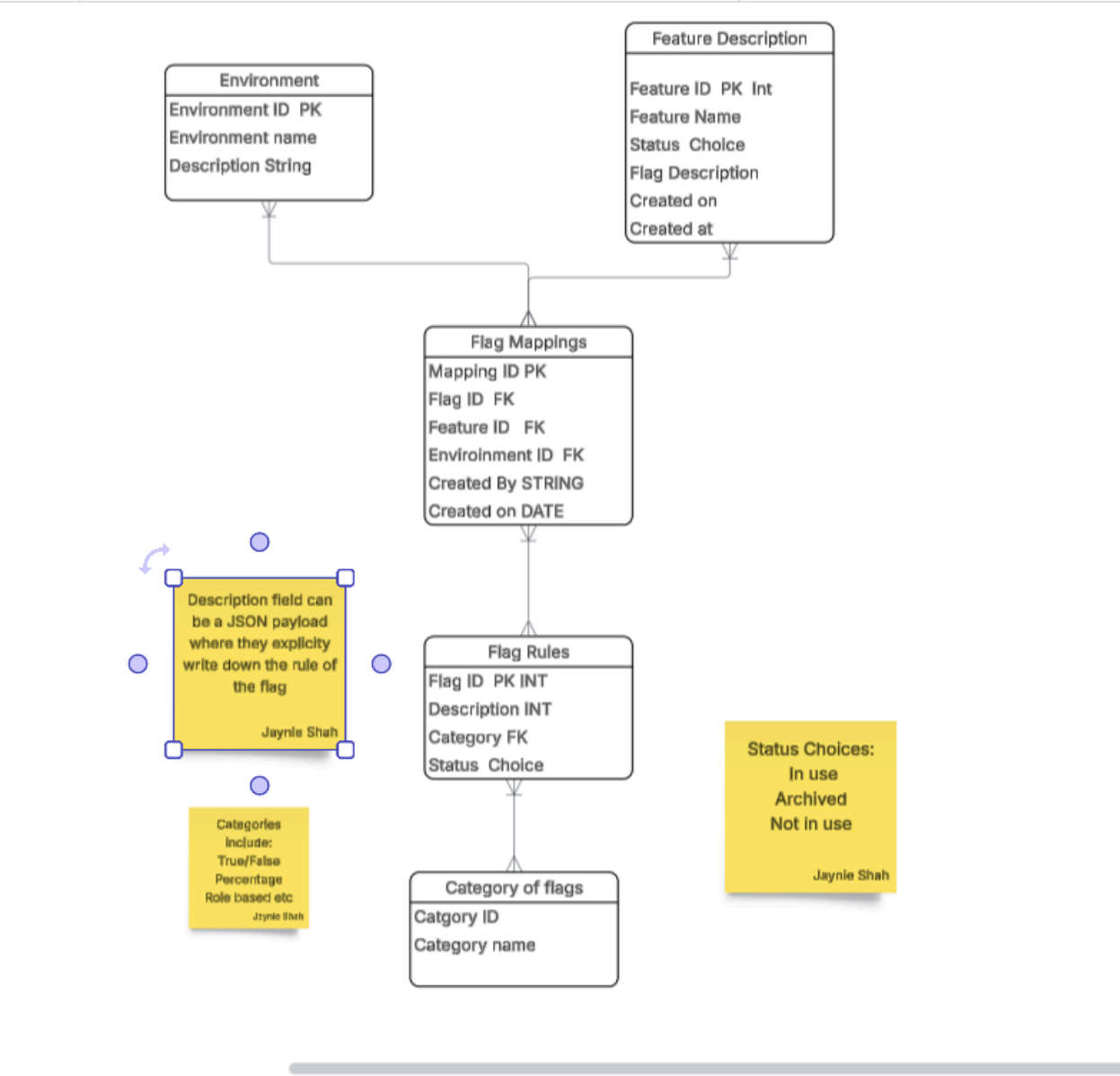
From what I read in the book, if a 'replica' fails (and in this case it's REDIS) , the REDIS database will have a log of all the write requests sent from the DB. The REDIS database must catchup as soon as its back online.

Arch diagram - high level





Data model



API specifications

Method	Purpose	Request	Response	Error
POST / PUT	Create requests from the admin user interface to	{"environment name" : Test, "Flag description":	200: Insert successful	404 = Bad request

	the database to manage flag mappings	{"user_ids": [123, abc, 324] , "Feature description" : "New_Feature101" "Created by		401 Not authorised
GET	Evaluate whether the user/group is allow access to a feature	"Feature Name: New_Feature101, "User ID: 7239"		

Load balancing Strategy

- A least-response-time load balancer should be used because some features may experience bursts of users, whereas others may receive regular requests at the same time.
- Trade off is that I will need all servers to have the same processing power and the servers need to track response times.

Archiving and approval process

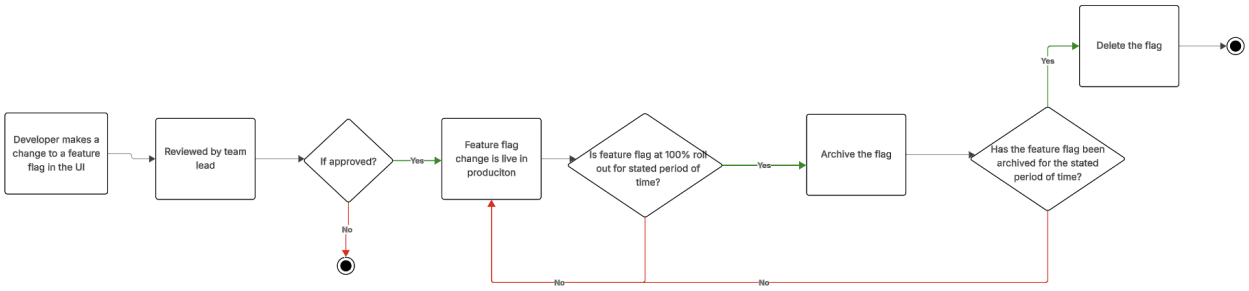
On the flag record, there is a archive column and a deletion column.

The archive and delete columns will indicate the number of days a column must be in the archive or deletion state before performing its relevant process.

The reason for this, over a standard archive/deletion process, is that certain features at 100% rollout might require a period of hyper care due to the volatility/size of the change. As such these columns will ensure that a clean-up process is always in place, even if it is dynamic for each feature.

There can be a daily job to check the dates of deletion. There isn't really that much of a performance trade off here because, a standardised archive/deletion

The slight performance trade off is that we have to have a job checking dates every day, rather than just an instant deletion as soon as there is 100% rollout



Rollback strategy

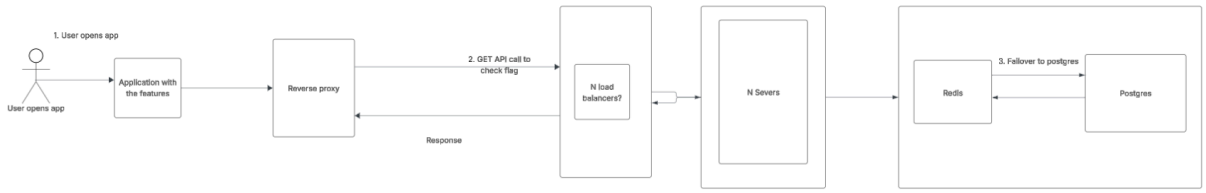
Having an active column on the flag will allow us to rollback the flag quickly, a developer can switch a flag to inactive and rollback the feature.

The risk here is that there is a backlog of events being processed and it will take long to deactivae

API vs SDK

An API would be suitable if the feature flag was being built in-house and simply internal applications would be using it.

As this is a feature flag that would be used by all sorts of organisations, I have a revised diagram below:



Future-proofing ideas (multi-tenant design, audit logs, SDK integration)

- If postgres and REDIS both fail, then default values must be set

Creating new users/groups

In ServiceNow, we have user criteria, which let you define access to certain resources. A user criteria record can store individual users, groups, roles , departments etc.

I would like to bring this idea to my feature flag system to prevent the need for having to hard code the target groups for each flag record. Rather we can reference this table and have reuse.

The assumption here is that we have tables for users, groups, roles, etc., and that the user criteria table can reference all of them.

4 Metrics & Dashboards

- Number of features we switch on/off
- Pie chart visualising each feature , showing the users and groups that the feature is on/off for
- Rate of adoption for each feature (i.e how long it took for 100% roll out of a feature)
- Rate of rollbacks that were caused by a feature release

Performance:

- Response time (Redis vs Postgres fallback)

Usage:

- Flag evaluations per flag (which flags are actually used)
- Evaluations per user/group

Edits:

- Flag changes (audit trail)
- Cache invalidation events

SDK vs Redis:

- Redis would enable scaling when there are applications on different languages
- I have traded off not having networking calls

Semi synchronous vs Fully async replication to Redis

- I would go with semi synchronous replication to avoid the situation where in
- The trade off here is that if there are multiple REDIS instances, we are likely

7 Potential Impact and Dependencies

- *What other systems or teams are affected by this proposal?*

Any application onboarding to the feature flag system will have to come up with processes to roll this out

The product management team will need to be informed about this because it will impact how they communicate roll out of features to stakeholders

- *How could this be exploited by malicious attackers?*
    - Attackers who gain access to the system could open up feature flags to the incorrect user groups
    - If there are bugs in the features, these could be exploited and it risks pre-releasing work that is not ready and is rushed
    - PII data could be set in the flags

8 Unresolved questions

- How could the database be expanded for teams with different purposes?
- Would strings

Notes: 11/02/26

- ~~Load Balancer → Multiple regions~~
- ~~Validation that there is a queue in case (in an event queue)~~
- ~~Fire wall~~
- ~~SDK vs API considerations?~~
- ~~Archiving vs Deleting Policies~~

- ~~Adding mechanisms for creating groups/users etc~~
- ~~Rollout strategies/approvals~~
- ~~Trade offs~~

Notes to complete 16/02/2026

- TTL / key values invalidation
- Caching mechanism
    - Justify the trade-off for the querying of the feature flag
- Consider read-only vs write-only to consider SQL/NoSQL
- Consider allocating resources geographically /database
- Circuit break
- Replication strategy
- Thinking about distributed data / propagating the data